

Description of Lab Work nº 2	
Course	Real-Time systems / <i>Sistemas de Tempo Real</i>
Year	2025/2026
Aim	Development of real-time applications in Java, leveraging concurrency, JNI, and DLL to manage an automated storage system that simulates the operations of an agri-food cooperative warehouse.
Classes	4 classes x 3 hours + 15 extra hours (outside)
Delivery Date	24/11/2025 - Via Moodle (single group submission).
Concrete Objectives: 1. Java language a. Characteristics of Java b. The “<i>Java Virtual Machine</i>” (JVM) c. Object-oriented programming d. Java training examples 2. Real-time features of Java a. The Thread class and the Runnable interface b. Synchronization and communication between Threads c. JNI for hardware access; DLL build & load-library procedure. 3. Develop a practical application of Java real-time as described in this document. 4. Solve the problem described in Description of Work using Java concurrency and native access through JNI/DLL.	

Description of Work

This work is aimed at providing the concepts of Java concurrency language features. For such, the development of a real-time solution for the problem described below is proposed. The work comprises the development of a program to perform the management of an automated **agri-food storage system**, capable of handling real-time events like incoming pallets, scheduled shipments, and environmental alerts (e.g., high humidity).

It comprises the hardware kit that represents the agri-food storage system, illustrated in fig. 1. To enhance students' performance other tools are available for developing this work, such as a storage simulator, fig.2 and a set of low-level functions for software/hardware interaction, provided in CLIP.

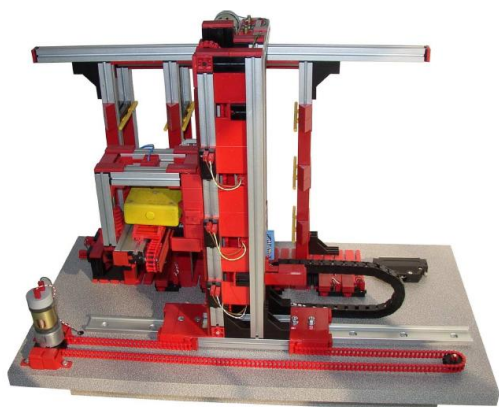


Fig. 1 –Automated Storage Kit

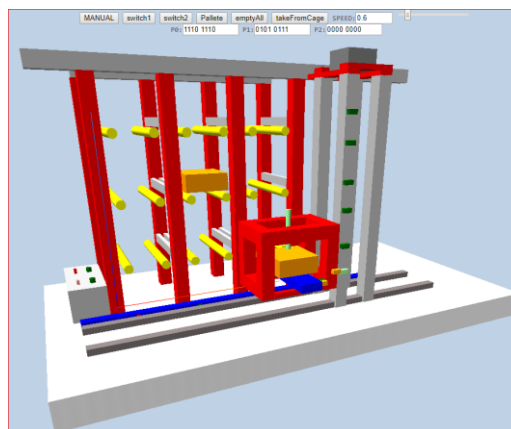


Fig. 2 – Automated Storage Simulator

The adequate solution for this problem requires the utilization of a set of technologies based on Java SDK, Java Native interface (JNI), and DLL development, with more emphasis on Java concurrency features.

The interaction between the real-time program and the automated storage is performed through an input / output digital data acquisition board, specifically, the NI USB 6509, as shown in fig. 3.



Fig. 3 – the NI USB 6509 data acquisition board

Work Context and Requirements

Agricultural cooperatives play a vital role in efficiently storing and distributing local production. During harvest seasons, multiple producers deliver pallets of crops such as maize, grapes, olives, and vegetables to shared storage facilities. Each batch has distinct properties: humidity level, harvest date, and destination (e.g., mills, wineries, distribution centers).

The cooperative must manage **batch arrivals**, limited storage capacity, and time-sensitive deliveries while maintaining traceability and safety. The goal of this lab is to implement an automated **agri-food storage system**, capable of handling real-time events like incoming pallets, scheduled shipments, environmental alerts (e.g., high humidity) and traceability.



Figure 1 – Agri-Food Storage System illustration. Generated with ChatGPT (OpenAI, GPT-5), November 2025.

The automated **agri-food storage system** should consider the following aspects:

Secção de Robótica e Manufatura Integrada

- The warehouse has 9 storage cells (3×3), each capable of **storing a single pallet**, so not all pallets can be stored.
- Pallets represent production batches (e.g., maize, grapes, olives).
- Each pallet includes metadata: product type, humidity (%), producer ID, harvest date, destination, and shipping date.
- The warehouse must automatically trigger shipping alerts, which are represented by LED 1 (at the left) flashing, when a batch's delivery date arrives or when humidity exceeds acceptable thresholds.
- Pallets with active alerts should be automatically scheduled for removal when Switch 1 is pressed. LED 1 (on the left) should continue to flash while the pallet removal process is underway.
- The “AI-Assisted Placement Module” (rule-based) recommends the most suitable cell for new pallets, aiming to balance occupancy, group products by type, and reduce distance.

Operation Modes

Mode 1 — Manager-Directed Placement/Withdrawal

- The warehouse manager manually specifies the storage cell (X,Z) for both storing and retrieving pallets.
- Placement (delivery): always performed at **(X=1, Z=1)** with **Y out (Y=1)**.
- Withdrawal (unloading): always performed at **(X=3, Z=1)** with **Y out (Y=1)**, meaning the cage moves the pallet to the far-right output position for pickup.

Mode 2 — System-Managed (Assisted) Placement/Withdrawal

- Placement: when no (X,Z) is specified, the “AI-Assisted Recommender” chooses the most suitable empty cell (based on balanced occupancy, grouping by crop type, and minimum travel distance). The placement still starts at **(X=1, Z=1)** with **Y out (Y=1)**.
- Withdrawal: unloading occurs at **(X, Z=1)**, where **X matches the column** of the pallet's storage cell.

Example: pallet at (**X=2, Z=3**) is unloaded at (**X=2, Z=1**).

- During Mode 2 withdrawal, from the moment a pallet is selected for removal until unloading is complete, **LED2 flashes** to indicate a dynamic column-aligned exit.

Movement constraints: motion along (X,Z) only with **Y=2** (center position); receiving/unloading only with **Y=1** (out position) at the designated entry/exit coordinate.

The functional **requirements** to develop the automated **agri-food storage system** are listed in the following table. They are split into basic or low-level requirements, for direct interaction with sensors and actuators, and high-level requirements, for storage management.

Table 1 – Agri-food Storage System Requirements

Req.	Description
	Low-level requirements (DLL development):
RL1	User Keyboard interaction providing execution of actions and showing warehouse states.
RL2	Develop the DLL and expose low-level I/O functions through JNI .
	High-level requirements (Java Development)
RH1	<u>Calibration</u> . Develop robust (semi-automatic) storage calibration. During a movement, the actuator must stop when it reaches a position sensor. It should always initiate with the cage in position (X=1, Z=1, Y=2).
RH2	<u>Manual Storage</u> . Store a pallet in the storage. Ask for pallet metadata (product_type, humidity, producer_ID, destination, shipping date) and target cell (X,Z). Validate if cell is empty.
RH3	<u>Assisted storage</u> . When (X,Z) is not provided, invoke the “AI-Assisted Placement Module” recommender (balance occupancy, group by type, minimize travel distance). Based on log decisions.
RH4	<u>Deliver pallet by cell (X,Z)</u> . Retrieve the pallet from cell (X,Z) specified through keyboard and unload based on the active mode: Mode 1: unload at (X=3, Z=1) with Y=1. Mode 2: unload at (X, Z=1) with Y=1 and LED2 flashing during the process.

RH5	<i>Deliver by product or producer.</i> List and deliver matching pallets.
RH6	<i>Shipping and humidity alerts.</i> Flash LED1 when a pallet's shipping date arrives or humidity exceeds a threshold (threshold may be defined by the student and must be documented).
RH7	<i>Mass removal (Switch 1).</i> Remove all pallets with active alerts; LED1 flashes until process completes.
RH8	<i>Emergency STOP (Switch 1 + Switch 2).</i> Emergency! Stop motion and freeze operations.
RH9	<i>Resume (Switch 1).</i> During an emergency STOP, if switch 1 is pressed, then RESUME operation.
RH10	<i>Reset (Switch 2).</i> During an emergency STOP, if switch 2 is pressed, then RESET the system (perform semi-automatic storage calibration).
RH11	<i>Emergency indicators.</i> Both LEDs flash rapidly (0.5 s) during emergency until resume/reset.
RH12	<i>Product list.</i> Display list of stored pallets (product types, producer IDs, humidity and shipping dates).
RH13	<i>Pallet lookup.</i> Display (X,Z), humidity, destination, and shipping info for selected product type/producer.
	Non-Functional requirements:
NF1	While operating, the system can accumulate requests .
NF2	While operating, the system can provide real-time information (occupancy, alerts) while maintaining safety rules (one pallet per cell, valid coordinates, Y = 2 for motion, Y = 1 for entry/exit).
NF3	The system always moves within limits X, Y, Z.
NF4	The system moves along (X,Z) only when Y is stopped at center position (Y=2) .

We remember that these requirements are better scored if their development is mostly based on the available JAVA concurrency features. For instance, behaviors in the system that run autonomously, should be programmed in separate threads; necessary interactions (synchronization / messages / ...) between the threads should rely on the concurrent API;

furthermore, access to exclusive access resources can be managed with classes containing synchronized methods.

As mentioned before, there is plenty of real-time behavior in the agri-food storage system, and as such, real-time approach should be adopted for developing the program that adequately operates the system. The real-time constraints presented in the storage system require the utilization of synchronization and communication between Threads, as well as handling the access to resources in exclusive mode. For an introduction on the utilization of threads, semaphores, communication, deadlock prevention, and so on, we recommend reading the tutorial available at

<http://download.oracle.com/javase/tutorial/essential/concurrency/index.html>.

Another important aspect of this work has to do with the interaction between JAVA and hardware. The kit representing the real system is connected via USB to the operating system (e.g., Microsoft Windows). On the other hand, Java programs run inside a Java Virtual Machine (JVM), which isolates Java programs from the Operating System and its resources. Therefore, the native machine and its resources, like the kit, are out of reach for these programs. To obtain access to them, it is necessary to provide Java Native interface (JNI). The functions for direct access to the hardware ports must be implemented inside a Dynamic Link Library (DLL). The construction of a DLL and its access from a Java program is described in the slides provided during the classes.

Classes Plan for the Lab Work 2

Class 1: [27 – 31 October 2025]

SW Installation & Interaction with “physical” system.
Small Java tutorial and low-level functionalities implementation.

Class 2: [03 – 07 November 2025]

JNI development for interacting with the physical system from Java.
Elaboration of both low-level and high-level routines for LabWork2.

Class 3: [10 – 14 November 2025]

Introduction to Threads and synchronization and communication mechanisms.
Development of the necessary threads for modeling LabWork2 concurrent behavior.

Class 4: [17 – 21 November 2025]

Fulfillment of each functional requirement towards the conclusion for LabWork2.

Professors:

- | | |
|---|----------------------------------|
| – Filipa Ferrada, fam@fct.unl.pt | (theoretical/laboratory classes) |
| – André Rocha, ad.rocha@fct.unl.pt | (theoretical/laboratory classes) |
| – Ana Inês Oliveira, aio@fct.unl.pt | (laboratory classes) |
| – Ricardo Peres, ra.peres@fct.unl.pt | (laboratory classes) |

Lab Work 2 Delivery

Students must submit via Moodle (**and only via Moodle**):

- ➔ Upload a zip/rar archive with your Visual Studio + Visual Code Projects.
- ➔ Upload a demo video (aprox. 5/8 minutes) presenting all implemented requirements/functionality. The video **must be** accompanied by a narration of what is being presented.

Evaluation criteria

It is provided (as a guiding object) a table with the project evaluation criteria. Each item will be quoted according to the approach and quality of the generated code.

ITEMS			QUOTATION/ PENALTY
FUNCTIONALITY	RL1	User Keyboard interaction providing execution of actions and showing warehouse states.	0.5
	RL2	Develop the DLL and expose low-level I/O functions through JNI .	1.0
	RH1	<u>Calibration</u> . Develop robust (semi-automatic) storage calibration. During a movement, the actuator must stop when it reaches a position sensor. It should always initiate with the cage in position (X=1, Z=1, Y=2).	1.0
	RH2	<u>Manual Storage</u> . Store a pallet in the storage. Ask for pallet metadata (product_type, humidity, producer_ID, destination, shipping date) and target cell (X,Z). Validate if cell is empty.	1.0
	RH3	<u>Assisted storage</u> . When (X,Z) is not provided, invoke the “AI-Assisted Placement Module” recommender (balance occupancy, group by type, minimize travel distance). Log each decision.	1.0
	RH4	<u>Deliver pallet by cell (X,Z)</u> . Retrieve the pallet from cell (X,Z) specified through keyboard and unload based on the active mode: Mode 1: unload at (X=3, Z=1) with Y=1. Mode 2: unload at (X, Z=1) with Y=1 and LED2 flashing during the process.	1.5
	RH5	<u>Deliver by product or producer</u> . Deliver matching pallets.	1.0
	RH6	<u>Shipping and humidity alerts</u> . Flash LED1 when a pallet’s shipping date arrives or humidity exceeds a threshold (at your	1.5

Secção de Robótica e Manufactura Integrada

		choice).	
	RH7	<u>Mass removal (Switch 1)</u> . Remove all pallets with active alerts; LED1 flashes until process completes.	1.0
	RH8	<u>Emergency STOP (Switch 1 + Switch 2)</u> . Emergency! Stop motion and freeze operations.	1.0
	RH9	<u>Resume (Switch 1)</u> . During an emergency STOP, if switch 1 is pressed, then RESUME operation.	1.0
	RH10	<u>Reset (Switch 2)</u> . During an emergency STOP, if switch 2 is pressed, then RESET the system (perform semi-automatic storage calibration).	1.0
	RH11	<u>Emergency indicators</u> . Both LEDs flash rapidly (0.5 s) during emergency until resume/reset.	0.5
	RH12	<u>Product list</u> . Display list of stored pallets (product types, producer IDs, humidity and shipping dates).	1.0
	RH13	<u>Pallet lookup</u> . Display (X,Z), humidity, destination, and shipping info for selected product type/producer.	1.0
	NF1	While operating, the system can accumulate requests .	1.0
	NF2	While operating, the system can provide real-time information (occupancy, alerts) while maintaining safety rules (one pallet per cell, valid coordinates, Y = 2 for motion, Y = 1 for entry/exit).	0.5
	NF4	The system always moves within limits X, Y, Z.	0.5
	NF5	The system moves along (X,Z) only when Y is stopped at center position (Y=2) .	0.5
QUESTIONNAIRE+ VIDEO + DISCUSSION	Functionality demo + question answers + others		2.5
TOTAL			20
No delivery of the development projects or the questionnaire or the video or missing discussion (ALL MANDATORY)			-20